PROJECT BRIEF · DRAFT V0.1

TIER

A native iOS health & weight-loss app that ranks you on improvement and consistency — not on absolute numbers.

> *"Every fitness app compares you to other people. This one compares you to who you were last month — and ranks you on that."*

PLATFORM	BUDGET	FIRST SHIP	DATED
Swift 6.3 · SwiftUI · iOS 18+	10–20 hrs / week	TestFlight · Nov 2026	27 July 2026

SECTION 00

How to Use This Brief

This document is built for handoff. Each phase is self-contained, with its own objective, technical detail, exit criteria, and a copy-paste prompt block for starting a fresh working session without re-explaining the project.

The three rules

1. **The thesis on the cover is the scope filter.** If a feature cannot be traced back to that sentence, it does not go in v1. Re-read it before every planning session.
2. **Do not skip exit criteria.** Each phase ends on a testable condition, not on a date. If the criteria aren't met, the phase isn't done — extend it and compress a later one.
3. **Decisions in §02 are locked.** Re-litigating settled decisions is the most expensive habit in solo projects. If a decision genuinely needs reversing, write down what new information changed it.

CONTEXT PRIMER — PASTE THIS TO START ANY NEW SESSION

```
I'm building "Tier", a native iOS health/weight-loss app in Swift 6.3 / SwiftUI, deployment target iOS 18.
Backend is Supabase (Postgres + Auth + Storage + Edge Functions). Solo developer, CS student, 10–20 hrs/week.

CORE THESIS: unlike every other fitness app, Tier ranks users on rate of improvement and consistency
relative to their own baseline — not absolute performance. This solves the Strava problem where
beginners are demoralised by leaderboards they can never win.

V1 SCOPE: food logging (text search / barcode / AI photo), weight logging with trend, HealthKit sync,
the ranking engine, weekly tier assignment. EXPLICITLY OUT OF V1: workout logging, AI coach chat,
social feed, recipes, meal planning, Android, Apple Watch.

I'm currently on PHASE [X]. [Paste the phase brief from the project document.]
Be direct and cynical about scope creep. Push back if I'm over-engineering.
```

Contents

01	The Thesis — and why it survives scrutiny	3
02	Decision Log (locked)	4
03	Scope — what's in, what's deliberately out	5
04	Technical Architecture	6
05	Phase 0 — Foundations	8
06	Phase 1 — Single-Player Core	11
07	Phase 2 — The Ranking Engine	13
08	Phase 3 — TestFlight & Real Users	15
09	Phases 4–5 — Social & Beyond	17
10	Costs, Risks & Safety	18
11	The Interview Narrative	20

SECTION 01

The Thesis

Absolute leaderboards punish exactly the people who most need a fitness app. Post a 12-minute mile on Strava and you're buried under people running sixes. Someone 25kg overweight will never top a “heaviest bench” board. The people with the furthest to go get the least encouragement, and they quit first.

Rate of improvement and consistency are things a total beginner can genuinely win at. That's the product insight. The convenient part is that these are also the two variables that most strongly predict long-term weight-loss outcomes — so the product design and the underlying science point in the same direction. That alignment is rare and worth building the entire app around.

Why this is defensible engineering, not just a nice idea

The app isn't the hard part. Calorie loggers are a solved problem. The hard part is designing a rating system that is simultaneously:

- **Fair across starting points.** 40kg → 60kg bench is +50%. 140kg → 150kg is +7% and vastly harder. A naive percentage-improvement score is nonsense.
- **Resistant to sandbagging.** Deliberately logging a weak first fortnight inflates your improvement curve for months.
- **Robust to self-reported garbage.** Every input is typed by a human with a direct incentive to lie once ranks exist.

> Prior art to stand on
> You are not inventing this from nothing. **Wilks and DOTS coefficients** in powerlifting already solved “how do you fairly compare a 60kg lifter to a 120kg lifter.” **Age-grading** in distance running solved the same problem across age. Both are published normalisation formulas with decades of scrutiny. Your contribution is adapting a well-understood normalisation problem to a new domain — which is a far stronger position than claiming novelty, and much easier to defend when someone technical interrogates it.

The honest cynicism

This category is saturated. MyFitnessPal, Lose It, Noom, Yazio and a wave of AI photo-loggers already exist, and health apps have famously brutal 30-day retention. “Calorie counter but gamified” is a pitch every hackathon judge has heard.

None of that makes this a bad project. It makes an undifferentiated version of it a bad project. The ranking system is the differentiator; everything else in the app exists to feed it data. If you find yourself spending three weeks perfecting a food search UI, you have drifted into building a worse MyFitnessPal.

SECTION 02

Decision Log

Locked decisions with their reasoning and accepted trade-offs. The point of writing these down is so you stop arguing with yourself at 1am in week six.

DECISION	REASONING	TRADE-OFF ACCEPTED
Native Swift / SwiftUI not React Native	Everyone in the immediate testing circle is on iOS. Native gets the richest HealthKit access, the smoothest feel, and the strongest "I shipped a native iOS app" signal.	Android is deferred, not free. Partial mitigation: Swift 6.3 shipped the first official Swift SDK for Android, so pure-logic packages — notably the ranking engine — can be shared later. UI would still need rebuilding.
iOS 18.0 deployment target	Unlocks the Observation framework, SwiftData and Swift Charts while keeping broad device coverage. Build with Xcode 26.6 (Swift 6.3) against the iOS 26 SDK — the App Store submission floor since 28 April 2026.	Excludes a small tail of older devices. Irrelevant at this stage.
Supabase backend	Real Postgres, not a document store. Ranking, leaderboards and cohort queries are inherently relational. Row-Level Security is the correct primitive for health data. Generous free tier.	Vendor lock-in on auth and edge functions. Acceptable — the data itself is portable Postgres.
Ranking engine ships client-side first	Zero latency, works offline, dramatically simpler while the only competitor is your past self.	**Must migrate server-side before leaderboards exist.** A client-authoritative score is trivially cheatable. This migration is the hard gate on Phase 4.
Fixed tier thresholds, not percentiles	Percentile ranking needs a user population you don't have. It also makes a user's rank move for reasons they didn't cause, which feels arbitrary and unfair.	Thresholds need manual re-tuning once real distribution data arrives in Phase 3.
Consistency weighted above progress	It's the strongest real-world predictor of outcomes and the hardest metric to fake — you cannot sandbag showing up for eight straight weeks.	Users chasing fast visible results may find early ranks feel unresponsive. Mitigate through copy, not by reweighting.
No workout logging in v1	Hevy-style set/rep/weight tracking is an entire application. It would double the build and halve the focus.	The app is a nutrition-and-consistency product at launch. Read workouts passively from HealthKit instead — you get the signal without the UI.

Reversal protocol

If you genuinely need to reverse one of these, write one line here stating *what new information* changed it. "I changed my mind" is not new information. Real triggers look like: a TestFlight metric, a technical blocker you hit, or an Apple policy change.

SECTION 03

Scope

In scope for v1

Sign in with Apple · Food search logging · Barcode scanning · AI photo logging · Correction & confirm flow · Weight logging + trend · HealthKit sync · Today dashboard · Ranking engine · Weekly tier assignment · Offline-first sync · Data export & deletion

Explicitly out of v1

~~Workout logging~~ · ~~AI coach chat~~ · ~~Social feed~~ · ~~Friend leaderboards~~ · ~~Push notifications~~ · ~~Recipes~~ · ~~Meal planning~~ · ~~Macro micro-optimisation~~ · ~~Apple Watch app~~ · ~~Widgets~~ · ~~Android~~ · ~~Web dashboard~~ · ~~Subscriptions / paywall~~

Read this when you feel the urge

Every item in that second list is a good idea. **That is precisely why they are dangerous.** Solo projects don't die from bad ideas, they die from too many good ones. Each of those features has a natural home in Phase 4 or 5, and each will be a better feature for being built on top of real usage data instead of guesses.

The tier ladder

Bronze	Silver	Gold	Platinum	Diamond
Starting out	Building habit	Consistent	Dialled in	Exceptional

Phase map

PHASE	OBJECTIVE	WINDOW	ENDS WHEN
0	Foundations & de-risking	3–16 Aug · 2 wks	Your own real health data renders on your own iPhone, and you have a photo-accuracy baseline
1	Single-player core	17 Aug – 13 Sep · 4 wks	You have personally logged 21 consecutive days of real data
2	The ranking engine	14 Sep – 11 Oct · 4 wks	Engine produces a rank you'd defend out loud; every exploit has a passing test
3	TestFlight & real users	12 Oct – 8 Nov · 4 wks	10+ real users and four weeks of retention data
4	Social layer	Nov – Dec	Server-authoritative scoring live; friend leaderboards shipped
5	Public launch & iterate	Jan 2027 →	Judgement call from retention data

The constraint nobody plans for

Final year starts late September. Phases 2 through 4 land on top of your degree. Assume your throughput **halves** from mid-September. The windows above already contain slack for this — do not spend it in August feeling productive.

SECTION 04

Technical Architecture

Stack

LAYER	CHOICE	NOTES
Language / UI	Swift 6.3, SwiftUI	Xcode 26.6 (current stable). **Do not develop on the Xcode 27 beta** for a project you intend to ship.
Architecture	MVVM with `@Observable`	Observation framework, iOS 17+. Not TCA — the learning tax is too high for a solo build and it will slow you down more than it helps.
Local storage	SwiftData	Local source of truth. Offline-first is non-negotiable — see below.
Backend	Supabase	Postgres, Auth, Storage, Edge Functions. `supabase-swift` SDK.
Auth	Sign in with Apple	Paid-programme entitlement. If you ever offer any third-party login, Apple requires you also offer Sign in with Apple — so start there and skip the rework.
Health data	HealthKit	Read: `bodyMass`, `stepCount`, `activeEnergyBurned`, `appleExerciseTime`, workouts.
Food database	Open Food Facts + USDA FoodData Central	Both free. Mirror into your own Postgres with a `tsvector` index — do not hit a third-party API on every keystroke.
Barcode	VisionKit `DataScannerViewController`	iOS 16+. Roughly twenty lines of code. Don't hand-roll AVFoundation.
Photo → food	Vision model via Supabase Edge Function	**Never call the AI API from the app.** Keys embedded in a client binary are extractable in minutes.
Charts	Swift Charts	First-party, no dependency.
Testing	Swift Testing	The modern replacement for XCTest.
Analytics	PostHog	Phase 3 onward. Free tier is ample.
Lint	SwiftLint	From day one, not retrofitted.

Layering

```
Views (SwiftUI)
  ↓
ViewModels (@Observable)
  ↓
Repositories (protocol-based)  ←  inject mocks here for testing
  ↓
Data sources: SwiftData (local) · Supabase (remote) · HealthKit (device)

RankingEngine — standalone Swift Package, pure functions, no I/O, no UI imports
```

Protocol-based repositories are what make the app testable, and being able to explain why is worth more in an interview than any individual feature. The ranking engine sits deliberately outside this stack: it takes data in and returns a score, touching nothing else.

Offline-first sync

> Why this matters more than it sounds
> Gyms have no signal. Restaurant basements have no signal. If logging fails offline, the user learns that the app is unreliable at exactly the moment they were building a habit — and they churn. This is one of the highest-leverage decisions in the whole build.

- SwiftData is the local source of truth. The UI never waits on the network.
- An **outbox table** holds pending mutations; a sync service drains it when connectivity returns.
- Last-write-wins using server timestamps for v1. Document the limitation rather than building CRDTs — knowing when *not* to over-engineer is itself a signal of judgement.

Core data model

TABLE	KEY FIELDS
`profiles`	id, created_at, dob, height_cm, sex, activity_level, goal_type, target_rate_kg_per_week, calibration_completed_at
`foods`	id, source (off/usda/custom), barcode, name, brand, kcal_per_100g, protein/carbs/fat, search_vector
`food_logs`	id, user_id, food_id, logged_at, grams, entry_method (search/barcode/photo/manual), **locked_at**
`photo_estimates`	id, log_id, storage_path, model_output (jsonb), confidence, **was_corrected**, correction_delta
`weight_logs`	id, user_id, measured_at, kg, source (manual/healthkit)
`health_samples`	id, user_id, date, steps, active_energy, exercise_minutes, workout_count, hk_anchor
`weekly_scores`	id, user_id, week_start, consistency, adherence, progress, composite, tier, engine_version, flags (jsonb)
`sync_outbox`	id, entity, entity_id, operation, payload, created_at, attempts

Design for social now, ship it later. Add `friendships` and `leaderboard_entries` to the schema in Phase 1 even though nothing writes to them until Phase 4. Retrofitting relationships into a live database with real user data is genuinely painful; adding two empty tables now costs you nothing.

Note `engine_version` on `weekly_scores`. When you retune the scoring weights — and you will — you need to know which version produced which historical score, or every past rank becomes meaningless.

Phase 0 — Foundations

3–16 August 2026 · 2 weeks · Objective: kill the risky unknowns before writing any real code

The purpose of this phase is not to build. It is to make sure that nothing in weeks 3–16 turns out to be impossible. Two things could sink this project — HealthKit access and photo-recognition accuracy — so both get proven in throwaway code first.

0.1 · Prerequisites — hardware & toolchain

REQUIREMENT	DETAIL
Mac	Apple Silicon strongly preferred. Must run **macOS Tahoe 26.2 or later** — this is Xcode 26.6's hard floor.
Disk space	Budget ~40GB for Xcode plus simulators. A real constraint on 256GB machines — check this before you start downloading.
Xcode	26.6 (stable, June 2026), bundles Swift 6.3. Since 28 April 2026 the App Store requires builds made with Xcode 26+ against the iOS 26 SDK.
iPhone	A **physical device** running iOS 18+. HealthKit does not meaningfully work in the Simulator — you cannot fake your way through this.
Deployment target	iOS 18.0

Buy the developer account in week one

HealthKit and Sign in with Apple are paid-programme entitlements. Free provisioning gives you 7-day certificates and no access to either — meaning you cannot even complete Phase 0's HealthKit spike without paying. Enrolment can also take several days to clear. This was originally scheduled for Phase 2; it moves to week one. Cost is $99/year plus 20% UK VAT for individuals.

0.2 · Accounts to create

SERVICE	COST	PURPOSE
Apple Developer Program	$99/yr + UK VAT	HealthKit, Sign in with Apple, TestFlight, App Store. **Week 1.**
GitHub	Free	Private repo, Actions for CI
Supabase	Free → ~$25/mo	Postgres, auth, storage, edge functions
Anthropic or OpenAI API	Pay-as-you-go	Vision model. Start with $5 of credit — it goes further than you'd think
Figma	Free	Design system and wireframes
PostHog	Free tier	Not needed until Phase 3, but create it now so you're not doing admin later

0.3 · Swift ramp-up

If Swift and SwiftUI are new to you, the fastest credible route is **100 Days of SwiftUI** (Hacking with Swift, free) or Stanford's CS193p lectures.

Do not complete the course before starting

Work through roughly days 1–30 *concurrently* with this phase, then learn the rest by building. Finishing a 100-day course before writing your own first line is the most common way this project quietly dies in September. You need enough Swift to be dangerous, not enough to be comfortable.

0.4 · Competitor teardown

Structured research, not vibes. Install and genuinely *use* each of these for one week: **MyFitnessPal** (the incumbent), **Cal AI** or similar (photo logging), **Hevy** (workout UX), **Strava** (the leaderboard problem you're solving), **Duolingo** (gamification done properly — not a fitness app, and that's the point), **Yazio** or **Zero** (onboarding and paywall patterns).

For each, record:

- **Taps from cold app open to a logged meal.** This is the single most important number in the category. MyFitnessPal is notoriously bad at it. Beating it is a real, measurable product claim you can put in a report.
- Onboarding length — how many screens before you can do anything useful
- Photo logging — is confidence shown? how do you correct a wrong estimate?
- Streak and gamification mechanics — what gets rewarded, what gets punished, how does it feel when you break one
- Paywall placement and what sits behind it
- The single most annoying thing about the app

Deliverable: a one-page comparison table plus a list of ten things you will deliberately do differently. This artifact goes into your final write-up and gives you concrete, specific answers when an interviewer asks how you approached product decisions.

0.5 · Figma — build a system, not screens

The failure mode here is designing forty beautiful screens you never build. You are a solo developer, not an agency. **Cap this at two days.**

1. **Tokens.** Colour palette including the five tier colours, type scale, spacing scale, corner radii.
2. **Components.** Button, text input, food row, stat card, tier badge, chart container. That's it.
3. **Wireframe exactly six screens:** onboarding/goal-set · today dashboard · log food (search + barcode + photo entry points) · photo confirm-and-correct · weight & trend · rank/tier screen.
4. **One high-fidelity hero screen — the rank screen.** It's the differentiator. It will be your App Store screenshot, your portfolio image, and the thing you show people when explaining the project. Make it good.

0.6 · Repo hygiene

- Private GitHub repo. `main` plus feature branches.
- Swift `.gitignore` from the github/gitignore templates — Xcode generates a lot of noise.
- **Never commit secrets.** API keys live in Edge Function environment variables, never in the app bundle or the repo.
- Conventional commits. Costs nothing, and it makes your git history itself a portfolio artifact that people genuinely do look at.
- SwiftLint configured now, while there are zero violations to fix.

0.7 · The two spikes

Throwaway code. Write it fast, learn what you need, then **delete it** — do not let spike code become the foundation of the real app.

SPIKE A — HEALTHKIT

Blank SwiftUI app → add the HealthKit capability → add `NSHealthShareUsageDescription` to Info.plist → request read authorisation for `bodyMass`, `stepCount`, `activeEnergyBurned` → print your last 30 days to the console. On a real device.

> The gotcha that catches everyone
> iOS deliberately will **not** tell you whether the user denied *read* access — this is an anti-fingerprinting measure, since knowing a user hid their heart data is itself information. You will get back an empty result set that looks identical to "this user has no data." Design "no data" as a normal, non-error state from the very beginning rather than treating it as a failure.

SPIKE B — VISION ACCURACY BASELINE

A script in any language: downscale a meal photo → send to the vision model with a structured-output prompt → return `{items:[{name, estimated_grams, kcal, confidence}]}`.

Run it against **twenty photos of your own real meals** where you know the actual values. Record the error rate.

This number is an asset

That measured baseline error rate is worth more than the feature itself. It gives you an honest accuracy figure for your write-up, a benchmark to improve against, and a genuinely impressive answer when someone asks “but how accurate is it really?” Most student projects cannot answer that question at all.

PHASE 0 EXIT CRITERIA

- Apple Developer Program membership active
- Your own real step and weight history rendering on your own iPhone
- Photo-accuracy baseline measured across 20 real meals and written down
- Figma tokens, component set, six wireframes, one hero screen
- Competitor comparison table plus ten deliberate differences
- Repo live, SwiftLint configured, secrets handling decided

PHASE 0 HANDOFF PROMPT

```
I'm on Phase 0 (Foundations) of Tier — de-risking before real development starts.
Working on: [HealthKit spike / vision accuracy spike / Figma design system / competitor teardown].
Environment: Xcode 26.6, Swift 6.3, iOS 18 deployment target, physical iPhone for testing.
Constraint: this is throwaway spike code — optimise for learning speed, not code quality.
```

Phase 1 — Single-Player Core

17 August – 13 September 2026 · 4 weeks · Objective: an app you personally use every single day

By the end of this phase the app does nothing clever. It logs food, logs weight, reads HealthKit, and shows you a dashboard. No ranking, no tiers, no social. This is the boring foundation that everything else stands on — and the phase where you become user zero.

1.1 · Authentication

Sign in with Apple via Supabase Auth. Enable Row-Level Security on *every* table from the first migration — retrofitting RLS onto a live database with real health data in it is a genuinely bad afternoon.

1.2 · Onboarding

Collect only what the scoring engine actually needs: date of birth, height, sex, activity level, goal type, target rate of change. Compute an initial calorie target from a standard equation (Mifflin-St Jeor is the usual choice) and present it as a **band, not a number**. Hitting exactly 1,847 kcal is not a real human goal; landing between 1,750 and 1,950 is.

> Safety gate — build this now, not later
> Onboarding is where the safety requirements in §10 are enforced: a hard minimum calorie floor, refusal to accept goal weights below a healthy BMI for the user's height, and an age gate. Building these in from the start costs a couple of hours. Retrofitting them after someone has been harmed costs considerably more.

1.3 · Food logging — three paths

PATH 1 — TEXT SEARCH

- Mirror Open Food Facts and USDA FoodData Central into your own Postgres. Index with `tsvector`.
- **Never** call a third-party API on every keystroke — it's slow, rate-limited, and fails offline.
- Surface recent and frequent foods first. Most people rotate through roughly thirty distinct foods; exploiting this makes your search feel dramatically faster than the incumbents' without any clever algorithm.

PATH 2 — BARCODE

VisionKit's `DataScannerViewController`, then an Open Food Facts barcode lookup. This is the fastest logging path that exists for packaged food and it is close to free to implement. Prioritise it.

PATH 3 — AI PHOTO

1. Capture or pick a photo
2. **Downscale to ~1024px on the longest edge before upload** — this materially cuts both cost and latency, and accuracy barely moves
3. Upload to Supabase Storage
4. Edge Function calls the vision model with a structured-output prompt
5. Return items with estimated ranges and a confidence level
6. **Mandatory confirm-and-correct screen**

> The photo rule — non-negotiable
> **Never silently accept an AI estimate.** Vision models miss cooking oil, hidden sugar and portion size; trained dietitians misestimate by 20–30% and the models do worse. You are not going to fix this, and pretending otherwise is how these apps lose users' trust permanently.
>
> Show a range. Show confidence. Make correction one tap. Then lean on the genuine insight: **for weight loss, a consistent 20% error still tracks the trend perfectly well.** What matters is the direction and the derivative, not the absolute number. Owning this limitation loudly is better product design *and* a far better interview answer than claiming you solved computer vision.
>
> Log every correction to `photo_estimates.was_corrected`. That dataset is how you improve the prompt later, and it is a real proprietary data asset that nobody else has.

1.4 · HealthKit integration

- Types: `bodyMass`, `stepCount`, `activeEnergyBurned`, `appleExerciseTime`, `HKWorkoutType`
- Use `HKAnchoredObjectQuery` together with `HKObserverQuery` and `enableBackgroundDelivery`
- **Persist the anchor.** This is the entire trick — it gives you incremental sync instead of re-querying full history on every launch, which is the difference between an app that opens instantly and one that hangs for four seconds.
- Workouts are read *passively*. You get training signal for the ranking engine without building any workout UI — which is exactly how the v1 scope stays sane.

1.5 · Weight tracking

Manual entry plus HealthKit reads (smart scales write there automatically). Display a **7-day exponentially weighted moving average — never raw daily weight.**

Daily weight is mostly water, glycogen and gut content. Showing it raw is actively harmful: it produces a jagged line that makes a genuinely successful week look like failure, and it is one of the main reasons people abandon weight-loss apps in the first fortnight. Swift Charts handles the visualisation.

1.6 · Today dashboard

One screen: calories remaining against the band, macro split, weight trend sparkline, consistency ring, and a placeholder where the tier badge will sit in Phase 2. Three tabs maximum across the whole app.

PHASE 1 EXIT CRITERIA

- **You have personally logged 21 consecutive days of real data.** Non-negotiable — this is the phase's actual purpose
- All three logging paths work, offline included
- HealthKit incremental sync working with a persisted anchor
- Weight trend rendering as a moving average
- Outbox sync verified by using the app in airplane mode
- RLS policies on every table, tested with two accounts

> **Why 21 days matters**
> You cannot design a fair ranking system without real data to design against, and Phase 2 depends entirely on having it. Your own three weeks of logs are the test set for the entire crown jewel of this project. This is also the point where dogfooding stops being a slogan: if *you* won't use it daily, nobody will.

PHASE 1 HANDOFF PROMPT

```
I'm on Phase 1 (Single-Player Core) of Tier. Building food logging, weight tracking, HealthKit sync,
and the Today dashboard. Architecture: SwiftUI + @Observable ViewModels + protocol-based repositories,
SwiftData local store with an outbox pattern syncing to Supabase.
Currently working on: [component].
Requirements: offline-first (the UI never waits on network), RLS on every table,
photo estimates always show ranges + confidence and require user confirmation.
```

Phase 2 — The Ranking Engine

14 September – 11 October 2026 · 4 weeks · Objective: the thing that makes this project yours

This is the crown jewel. Everything before it was plumbing; everything after it is distribution. If you have four good weeks in this entire project, spend them here.

2.1 · Where it lives

A standalone Swift Package called `RankingEngine`. **Pure functions. No I/O. No SwiftUI or UIKit imports.** Data in, score out. This isolation is what makes it exhaustively testable, trivially portable, and genuinely showable to someone in an interview.

The architectural decision worth articulating

Client-side scoring is entirely fine while the only person you compete with is your past self. **The moment leaderboards exist, it must move server-side** — a client-authoritative score is trivially cheatable with a patched binary, and a competitive system where cheating is easy is worse than no competitive system at all.

So keep the engine deterministic and pure, with test vectors that will validate an identical server-side port. Then in an interview you get to say: *"I ran scoring client-side for latency and offline support, but I knew it had to become server-authoritative before any competitive feature shipped, so I designed the module to be portable from day one."* That is a materially more senior thing to say than describing any feature you built.

2.2 · The maths

$$S = w_c \cdot C + w_a \cdot A + w_p \cdot P$$

C = consistency · A = adherence · P = normalised progress

Starting weights: $w_c = 0.50$ $w_a = 0.25$ $w_p = 0.25$

C — CONSISTENCY

Days logged divided by seven, but with a **floor on what counts as "logged."** Require at least one meal *and* a plausible fraction of estimated maintenance calories. Without this floor, users farm streaks by logging a single apple at 11:58pm, and your headline metric becomes meaningless within a fortnight.

A — ADHERENCE

The fraction of days where intake landed inside the target band. A band, not a point — see §1.2.

P — NORMALISED PROGRESS

This is the genuinely interesting part, and the part worth being able to explain properly.

Expected weekly weight change is a function of starting body mass and deficit. Someone at 120kg loses faster than someone at 70kg on an identical relative deficit. That is **physiology, not effort** — and rewarding it would exactly reproduce the unfairness you set out to fix.

- Score progress against a *predicted* trajectory for that individual's starting point: `P = actual_change / expected_change`, clamped to a sane range.
- This is precisely the DOTS/Wilks logic — a coefficient that makes different bodyweights comparable.
- Apply diminishing returns through a log or sigmoid curve so that early, easy losses don't dominate the score for months.

TIERS

Map S onto Bronze → Silver → Gold → Platinum → Diamond. **Fixed thresholds in v1**, revisited once Phase 3 gives you real distribution data. Recalculate weekly.

The retention mechanic

The weekly rank-up is your core retention moment. **One scheduled, emotionally meaningful notification per week beats twelve nagging ones** — and it respects the user, which the incumbents mostly don't. Resist the urge to add daily nudges; they train people to disable notifications entirely, and then you've lost the channel permanently.

2.3 · Anti-gaming

Five rules, in rough order of value for effort:

RULE	MECHANISM
Calibration window	The first 14 days establish a baseline, with outlier rejection (drop the extremes). No progress score is computed until it closes. This is the primary defence against sandbagging.
Retroactive edit lock	Logs older than 48 hours cannot be edited (`food_logs.locked_at`). One line of business logic that eliminates an entire class of exploit — retroactively "improving" a bad week.
Score delta cap	Cap week-over-week score movement. Blunt, unglamorous, extremely effective.
Weight passive data	Steps and active energy come from the motion coprocessor and are far harder to fabricate than a number typed into a text field. Weight them accordingly in the composite.
Plausibility flags	Z-score thresholds on weight change and intake. **Flag, don't silently reject.** Over-aggressive rejection destroys trust with honest users who happen to be outliers — and there are always more of those than cheats.

2.4 · The test suite *is* the deliverable

Write an explicit, named, failing-then-passing test for each exploit:

- Sandbagged calibration window
- Single-day binge distorting a weekly average
- Weight spoofing — entering a fake high starting weight
- Streak-farming with minimal logs
- Retroactive editing of a bad day
- Delete-and-recreate to erase a bad day
- Timezone abuse — logging across boundaries to claim extra days

Use Swift Testing. Consider property-based tests for invariants such as "score is monotonically non-decreasing in consistency, holding all else equal." Then **backfill the engine across your own Phase 1 data** and ask the only question that matters: does the rank it gives you feel *right*? Tune until it does, and write down what you changed and why.

PHASE 2 EXIT CRITERIA

- `RankingEngine` is a standalone package with zero UI dependencies
- Every exploit above has a named test that fails against a naive implementation and passes against yours
- Engine backfilled over your own 21+ days of real data, and the output feels defensible
- Tier badge and weekly rank screen live in the app
- A written note explaining every weight and threshold you chose — this becomes your report and your interview script

PHASE 2 HANDOFF PROMPT

```
I'm on Phase 2 of Tier — building the ranking engine, which is the core differentiator.

It's a standalone Swift Package: pure functions, no I/O, no UI imports, fully unit tested with Swift Testing.
Weekly composite: S = 0.5*consistency + 0.25*adherence + 0.25*normalised_progress.
Progress is normalised against expected change for the user's starting bodyweight (DOTS/Wilks-style
coefficient logic) with diminishing returns applied.

Anti-gaming rules: 14-day calibration window with outlier rejection; 48h retroactive edit lock;
week-over-week delta cap; passive HealthKit data weighted above typed data; z-score plausibility flags.

Currently working on: [component]. Be adversarial — try to find ways to game whatever I show you.
```

Phase 3 — TestFlight & Real Users

12 October – 8 November 2026 · 4 weeks · Objective: find out how wrong you were

This phase exists to break your assumptions. Treat bad feedback as the deliverable — a phase that produces no unpleasant surprises has failed, because it means you weren't testing with real people doing real things.

3.1 · TestFlight mechanics

TRACK	LIMIT	REVIEW	USE FOR
Internal	100 testers	None — live within minutes	**Your route for friends.** Testers need App Store Connect user access, which is a minor hassle, but there's no review wait at all
External	10,000 testers	Beta App Review on the first build of each version (~24h)	Later, if you go beyond your immediate circle. Public links available

Builds expire after 90 days

When a TestFlight build expires, testers lose the app entirely — **including local data and login state.** Plan a rebuild cadence and warn testers before it happens. Silently wiping someone's six weeks of food logs is a spectacular way to lose your only users.

3.2 · Before you invite a single person

- **Privacy policy live.** GitHub Pages is free and entirely adequate.
- **UK GDPR compliance.** Health data is *special category data* under Article 9 — it requires explicit consent and a documented lawful basis. You need a working data export and a "delete my account and everything in it" button that genuinely deletes. This is not optional once real users exist, and building it is a day's work at this stage versus a nightmare later.
- **App Privacy details** ("nutrition labels") completed accurately in App Store Connect. Under-declaring health data collection is a fast route to rejection.
- Crash reporting wired up — Xcode Organizer at minimum, Sentry if you want detail.
- PostHog SDK integrated and verified.

3.3 · Instrument before you ship, not after

Decide your metrics now. Retrofitting analytics means throwing away your first fortnight of data — the most interesting fortnight you'll ever have.

METRIC	WHY IT MATTERS
Time from install to first logged meal	Your onboarding friction, measured. Compare it directly against the competitor teardown numbers from Phase 0
D1 / D7 / D28 retention	**The only number that decides whether this is real.** Everything else is diagnostic
Log method split	Search vs barcode vs photo. Tells you where to invest — and may well surprise you
Photo correction rate	How often users override the AI. Your accuracy proxy in the wild, and a direct check on the Phase 0 baseline
Weekly rank screen open rate	Is the core differentiator actually landing?
Churn point	The last screen a user saw before they stopped coming back. Often the single most actionable thing you'll learn

3.4 · Feedback loop

TestFlight has **shake-to-feedback with automatic screenshot attachment** built in. Almost nobody knows this exists — tell your testers explicitly and you'll get several times more feedback than you otherwise would.

Beyond that: a group chat for informal reports, and **three short one-to-one conversations in week two**. Not a survey. Watch someone log a meal without helping them and you will learn more in five minutes than from fifty survey responses.

PHASE 3 EXIT CRITERIA

- 10+ real users who are not you
- Four weeks of retention data
- A written list of everything you got wrong — **this is the actual deliverable**
- Real tier distribution data, used to retune the thresholds from Phase 2
- An honest judgement recorded: is D28 retention good enough to justify Phase 4?

The number that decides everything

Of your ten testers, how many are still logging in week four? **If it's two, no amount of marketing or extra features fixes that** — and you will have learned something genuinely valuable about the category, which is a perfectly respectable outcome for a project like this. If it's seven, you have something real and Phase 4 is worth every hour.

Either way you get a strong write-up and a strong interview story. The failure mode is not "low retention" — it's never measuring, and therefore never knowing.

PHASE 3 HANDOFF PROMPT

```
I'm on Phase 3 of Tier — TestFlight beta with real users. App is feature-complete for v1
(food logging, weight tracking, HealthKit, ranking engine, tiers).

Working on: [App Store Connect setup / privacy policy / GDPR compliance / analytics instrumentation /
onboarding polish / interpreting retention data].

Key metrics: time-to-first-log, D1/D7/D28 retention, log method split, photo correction rate,
rank screen open rate, churn point. Distribution via TestFlight internal testing (100 tester limit,
no review). UK GDPR applies — health data is Article 9 special category data.
```

Phases 4 & 5 — Social and Beyond

November 2026 onward · Deliberately less detailed — it should be rewritten using what Phase 3 teaches you

Phase 4 — Social layer · Nov–Dec 2026

The cold-start problem is now solved, because Phase 3 gave you actual users who know each other.

Hard gate

Migrate scoring server-side before any leaderboard ships. Not after, not alongside. The moment ranks are competitive, a client-authoritative score is an open invitation, and a leaderboard everyone knows is cheatable is worse than having none. The pure-function design from Phase 2 is what makes this port straightforward — port the module, run the identical test vectors against both implementations, verify they agree on your real historical data, then cut over.

- Friendships, friend leaderboards **ranked on relative tier, never on absolute weight or volume** — the entire point of the thesis
- Push notifications for the weekly rank moment (one per week — see §2.2)
- Streaks, with recovery mechanics rather than punishment framing
- An always-available switch to disable every competitive and social feature (§10)

Phase 5 — Public launch and iterate · Jan 2027 onward

- Public App Store release
- Workout logging, if and only if the data says users want it
- AI coach — but grounded in the user's own logged history via retrieval, not a generic chat wrapper. This is the difference between a real feature and a demo
- Widgets, Apple Watch, Android via the Swift Android SDK
- An honest strategic decision, made from retention data rather than sunk cost

SECTION 10

Costs, Risks & Safety

Running costs

ITEM	COST	NOTES
Apple Developer Program	$99/yr + 20% UK VAT	Unavoidable. Required from week one for HealthKit
Supabase	£0 → ~$25/mo	Free tier covers Phases 0–3 comfortably
Vision API	~$0.01–0.03 / image	See below — this is the one that can surprise you
PostHog	£0	Free tier is generous
Domain (optional)	~£10/yr	GitHub Pages hosting is free
Through Phase 3	**~£130–170 total**	Plus your time, which is the real cost

Vision API cost control

Twenty users logging three photos a day is roughly 1,800 calls a month — real money at a few cents each. Mitigate by downscaling aggressively before upload, caching by image hash (people photograph the same meal repeatedly), and putting a per-user daily cap in the Edge Function. Add the cap *before* you hand the app to anyone, not after a surprise bill.

Risk register

RISK	SEVERITY	MITIGATION
Disordered eating harm	**Critical**	See safety requirements below. Non-negotiable and built in from Phase 1
Scope creep	**High**	The cover thesis is the filter. §03 lists what's out and why. Re-read both weekly
Motivation wall (~week 8)	**High**	The single most common killer. Dogfood daily from Phase 1; Phase 3 friends create genuine external accountability
Term-time collision	**High**	Phases 2–4 overlap with final year. Assume throughput halves from mid-September. Slack is already built in — don't spend it early
Building the app instead of losing weight	**Medium**	Genuinely a documented failure mode of this exact project. Dogfooding is both the mitigation and the point
App Store rejection	**Medium**	Health apps face stricter review. No medical claims anywhere in copy. Clear HealthKit justification. Accurate privacy labels
UK GDPR non-compliance	**Medium**	Article 9 special category data. Explicit consent, working export and deletion, before the first external tester
Photo accuracy disappointment	**Medium**	Unfixable. Manage it through honest UX — ranges and confidence, never false precision
Vision API cost overrun	**Low**	Downscale, cache by hash, per-user daily caps
Solo bus factor	**Low**	This document. Keep it current — it's the reason a new session can pick up where you left off

Safety requirements — read twice

This is the most important page in the document

A gamified calorie app with streaks and competitive ranks can **actively reinforce disordered eating.** This is a real, well-documented harm. Apple has rejected apps over it. And you will have real users, some of whom will be vulnerable in ways you cannot see from a dashboard.

Every one of the following is built in from Phase 1, not bolted on before launch:

- **Hard minimum calorie floor.** The app refuses to set a target below it, regardless of what the user requests.
- **Goal weight validation.** Reject target weights below a healthy BMI for the user's height.
- **No punishment framing.** "You broke your streak" becomes "welcome back." Losing a tier is framed as a reset, never as failure. Review every string in the app against this.
- **Age gate.** No under-18s in v1. The safeguarding considerations for minors and body-image content are substantial and well beyond what a solo v1 should attempt.
- **A visible, always-available switch** to disable all competitive and social features, without disabling the app's core usefulness.
- **Signposting.** Where patterns suggest it, surface support resources — in the UK, the National Alliance for Eating Disorders and Beat are appropriate starting points.
- **Rate-of-loss ceiling.** The engine should not reward losing weight faster than a safe rate. Score the *appropriate* rate highest, not the maximum — this is a place where the ranking design and the safety design happen to align perfectly.

This is the right thing to do independent of any other consideration. It also happens to be a genuinely rare and impressive thing to be able to discuss — "I thought hard about the harm my product could do and designed against it, here's specifically how" puts you in a very small minority of student projects.

SECTION 11

The Interview Narrative

Do not lead with “I built a calorie counter.” That invites a shrug and a change of subject. Lead with the problem, because the problem is what makes the engineering interesting.

> *“Fitness apps rank people on absolute performance, which demotivates exactly the beginners who most need them. I built a rating system that scores relative improvement and consistency instead. The hard part was making it fair across wildly different starting points and resistant to gaming — I adapted the normalisation approach powerlifting uses for the DOTS coefficient, and built a test suite specifically targeting exploits like sandbagging the calibration window.”*

Four deep-dives from one project

THREAD	WHAT YOU CAN GO DEEP ON
Algorithm design	The normalisation maths, the diminishing-returns curve, why consistency is weighted at 0.5, the DOTS/age-grading precedent, and what you'd change with more data
Adversarial thinking	The anti-gaming test suite, the client-to-server authority migration, why you flag rather than reject, and the exploits you found by attacking your own system
Product judgement	Real retention numbers, the competitor teardown, what you deliberately cut and why, the honest photo-accuracy baseline, and what the data told you that you didn't expect
Ethics & safety	Disordered eating mitigations, GDPR Article 9 handling, and why the rate-of-loss ceiling is both a safety feature and a scoring feature

The two things that make this beat other student projects

1. **Real users and real numbers.** Almost every student portfolio project has zero users and no measurements. Ten real testers and a D28 retention figure — even a bad one — puts you in a different category entirely, because it means you shipped and then faced the consequences.
2. **A defensible thesis rather than a feature list.** Feature lists are forgettable. A clear, arguable position that you can defend under questioning is what people actually remember about a candidate.

One last piece of cynicism

The most likely outcome is not that this fails technically. You will almost certainly be able to build it. The most likely failure is that it stalls in week eight, when the novelty is gone, term has started, and no one is waiting for the next build.

Everything structural in this plan — the 21-day dogfooding requirement, the early TestFlight date, getting real friends onto it before the social layer exists — is designed against that specific failure, not against technical risk. **Ship something to real people early enough that other people's expectations start doing the motivating for you.**